

Session Hijacking: Bypassing Multi-Factor Authentication

by

Anthony Costa Jr, Cayden Evans, Quinn Kneeland, & Nolan Pieters

Department of Electrical and Computer Engineering and
Computer Science
Tagliatela College of Engineering
University of New Haven

ABSTRACT

Session hijacking is a security threat where attackers are able to intercept or manipulate an active user's session to gain unauthorized access to systems or accounts belonging to a system. While there are many different security measures that can be implemented to mitigate the threat of these attack types, Multi-Factor Authentication (MFA) has become a widely implemented and industry standard approach to enhancing security. However, MFA still remains vulnerable to session hijacking, particularly after session tokens are stolen post-authentication. This report examines the weaknesses of current MFA security implementations against session hijacking and explores potential countermeasures to address the threat.

The objective of the research is to analyze the techniques and attack strategies used in session hijacking, assess the limitations that MFA remains prone to in mitigating these attacks, and, finally, propose defense mechanisms to increase overall session security. The methodology includes an encompassed review of existing session hijacking techniques, proof-of-concepts validating the presence of the issue, and development of a prototype solution aimed at detecting and preventing unauthorized session access.

The findings and conclusions indicate that while MFA effectively protects initial authentication, it does not safeguard active sessions, which ultimately leaves users vulnerable to attacks surrounding the theft of session token. The report proposes an enhanced security framework that incorporates session monitoring and token binding to

detect security threats and anomalies in a user's session activities. The implementation of a browser extension coupled with a backend database aided in the creation of a model security solution that effectively combat session hijacking attacks.

The results emphasize the need for implementing authentication mechanisms other than MFA to protect against session hijacking attacks. Additionally, the implications emphasized within the report show the necessity for organizations to adopt more advanced session protection strategies to mitigate the risk of hijacked sessions.

TABLE OF CONTENTS

	Page
LIST OF FIGURES.....	vi
Chapter(s)	
I. INTRODUCTION	
Introduction.....	1
Problem Statement.....	2
Project Objectives.....	2
Limitations of the Project.....	3
Definition of Terms.....	3
II. BACKGROUND AND REVIEW OF LITERATURE	
Introduction.....	6
Background Related to the Project.....	6
Literature Related to the Project.....	7
III. METHODOLOGY	
Introduction.....	12
Design of the Project.....	13
Tools and Technology used.....	16
IV. DISCUSSION AND RESULTS	
Introduction.....	18
Results and Discussion.....	18
V. CONCLUSIONS AND FUTURE WORK	
Conclusions.....	22

Future work	22
REFERENCES.....	24

LIST OF FIGURES

Login Flow with MFA



Login Flow with MFA After Session Hijacking Attack



Figure: 1
Page: 13

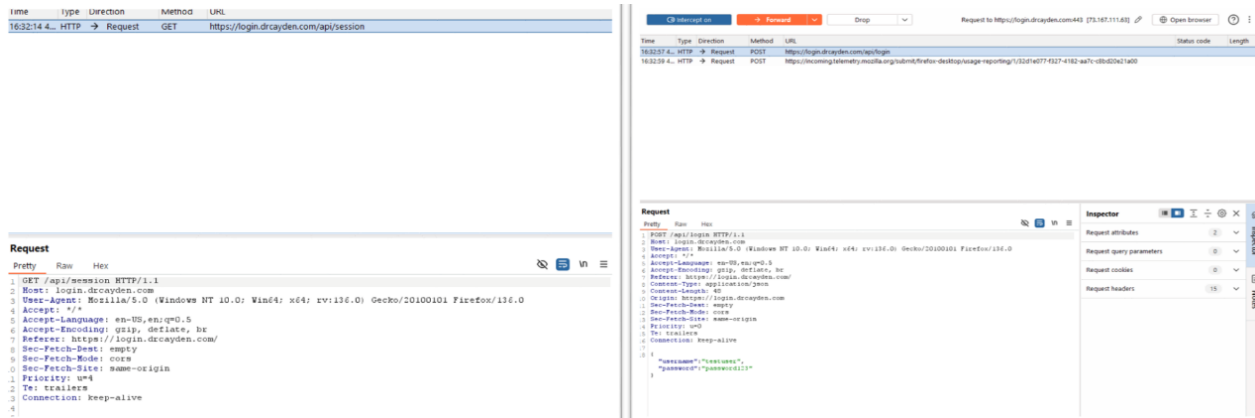


Figure: 2
Page: 13

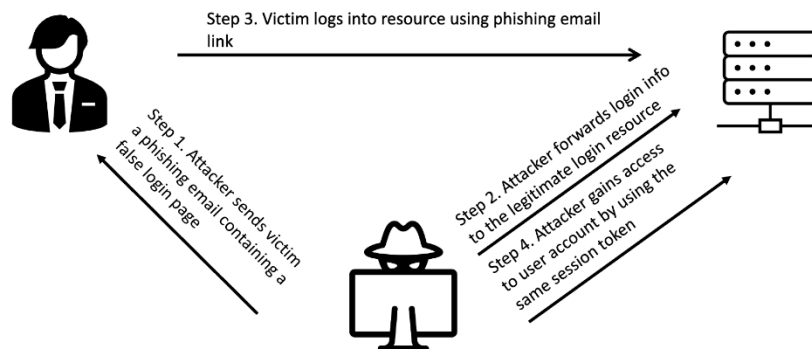


Figure: 3
Page: 13

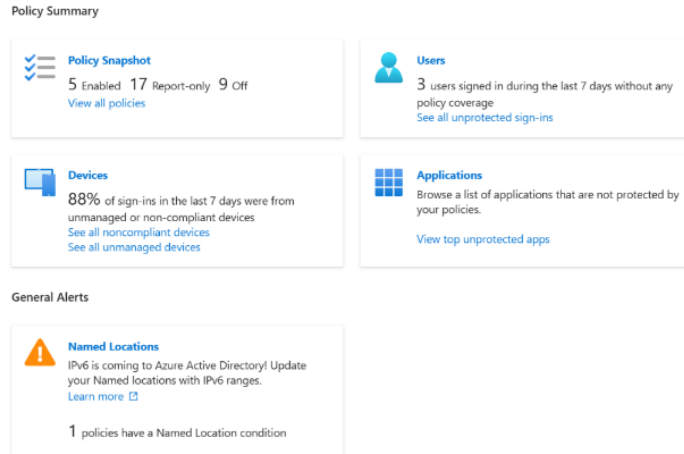


Figure: 4
Page: 14

id	username	password_hash	mfa_secret	mfa_enabled	browser_key	created
1	testuser	2y1238tCwNpt4zPdILLJ/Sqbg01KaI7tsHouVm1.dcrNR3HTScvk1pQku	KZXW6YTBMF2GQZLTUQUHI	1	9e7a61c4-d882-4c9f-b19a-9bcfd0c9aab	2025-05-02 16:23:54

Figure: 5
Page: 14

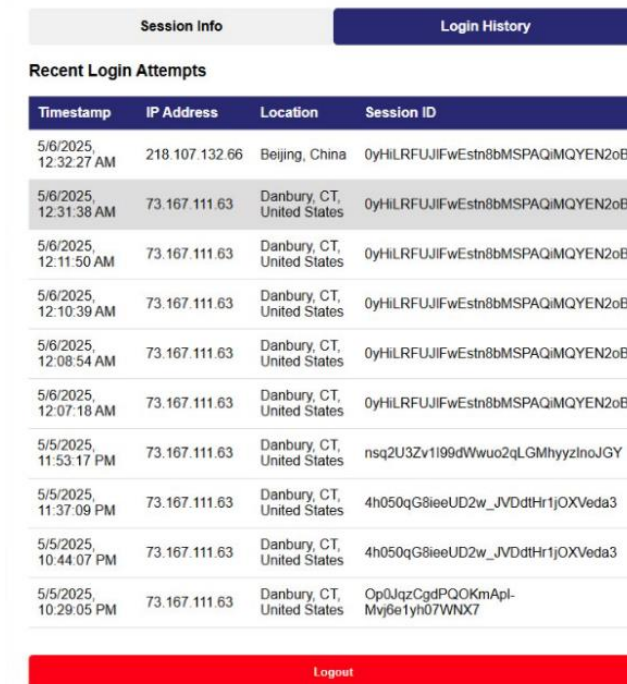


Figure: 6
Page: 19

Chapter I

INTRODUCTION

Introduction

This report analyzes the effectiveness and impact of multifactor authentication (MFA) in preventing session hijacking attacks. Session hijacking, a cyber-attack where an adversary attempts to gain unauthorized access to a user's active session, remains a constant security threat in the modern cybersecurity landscape, even in systems employing MFA. The analysis examines common vulnerabilities within MFA implementations that can be exploited, particularly regarding methods attackers may use to intercept or exploit active sessions, despite the added security layers MFA supposedly adds.

Through modern research on the topic of session hijacking, as well simulated attack scenarios, the study assessed how session hijacking can occur in instances with the protection provided by MFA. The findings reveal specific weaknesses in MFA protocols and measures, clearly showing how certain attack vectors can ultimately bypass the intended security. Recommendations are provided to strengthen defenses against session hijacking, emphasizing strategies to address and mitigate any identified vulnerabilities and generally enhancing the overall resilience of systems that rely on MFA.

Problem Statement

The project was undertaken to address the growing concern that multifactor authentication (MFA), sometimes unknowingly to nonpractioners, provides to systems. Despite its more recent widespread usage as a security measure, MFA still remains vulnerable to session hijacking attacks, which can ultimately compromise user accounts and sensitive data, and lead to other unintended consequences. The persistence of these attacks indicated potential weaknesses in current MFA implementations that required further investigation, and potential remediation to mitigate vulnerabilities in this widespread security implementation.

Project Objectives

1. Understand how session hijacking occurs
2. Perform a simulated session hijack against a known domain and identify weaknesses using a domain with MFA implementation.
3. Identify weaknesses in OAUTH Multifactor Authentication (MFA).
4. Identify protection measures to mitigate and/or prevent session tokens from being used from different systems.
5. Test the implementations of all findings to ultimately increase the security posture of any system implementation utilizing MFA.

Limitations of the Project

Many limitations identified throughout the development of the prototype solution were the implementation of the protection measures. The methodology behind how to

best fit the framework of the solution was ever-changing and required constant optimization to best fit the needs of potential users.

Additionally, the testing and use-case used in the creation of the prototype solution was restricted to a locally hosted server, as many of the testing mechanisms that we used were primarily enterprise solutions that would administer an abuse complaint, and therefore not be an effective long-term option.

Definition of Terms

Access Control: A security technique that restricts access to systems or resources based on user identity and roles, typically enforced through policies and permissions.

Attack Vector: A method or pathway used by an attacker to infiltrate a system or network, such as phishing, malware, or exploiting software vulnerabilities.

Cross-Site Scripting (XSS): A web security vulnerability that allows attackers to inject malicious scripts into content from otherwise trusted websites, often used to steal session tokens.

Impossible Travel: The detection of a user's account activity originating from two geographically distant locations within a timeframe that is impossible for a person to travel between them.

Man-in-the-Middle (MitM) Attack: A type of cyber-attack where the attacker secretly intercepts and potentially alters the communication between two parties, often used to steal session tokens or credentials.

Multifactor Authentication (MFA): A security process that requires users to provide two or more verification factors (such as a password, a fingerprint, or a security token) to gain access to a system, aiming to enhance protection beyond just a single password.

OAuth: An open standard for access delegation, used as a way to grant websites or applications limited access to user information without exposing the user's credentials.

Replay Attack: A type of network attack where valid data transmission (e.g., session tokens or authentication requests) is maliciously or fraudulently repeated, allowing attackers to impersonate a legitimate user.

Session Hijacking: A type of cyber-attack where an attacker takes control of a user's active session by stealing or intercepting session tokens or cookies, allowing unauthorized access to the user's account.

Session Token: A piece of data generated by a server and provided to a user after successful authentication, used to identify and maintain the user's session across multiple interactions without re-authenticating.

Token Replay: A specific form of session hijacking where an attacker intercepts a session token and reuses it to gain unauthorized access to an active session.

Chapter II

BACKGROUND AND REVIEW OF LITERATURE

Introduction

This chapter will explain the foundational concepts, background, and prior research on session hijacking and its methods of prevention. Additionally, an in-depth analysis of the literature was used to reinforce the concepts discussed will be presented

with a clear emphasis on how the research relates to session hijacking and our project's goals and parameters.

Background Related to the Project

Session hijacking is a significant cyber threat where attackers exploit vulnerabilities in session management and access control to gain unauthorized access to active user sessions without needing to verify their identity. This attack can lead to account takeovers, unauthorized data access, and many other forms of cyber exploitation. Despite advancements in authentication technologies, like multifactor authentication (MFA), session hijacking remains a persistent threat with few protections.

MFA is designed to strengthen access security by requiring users to verify their identity through multiple factors, such as a password, a security token, and/or a biometric verification. By requiring the use of multiple forms of authentication, it is harder for attackers to gain access to accounts without extensive knowledge of the target and even physical access to them or their devices. However, the added layers of protection are not immune to exploitation. Attackers have employed more sophisticated methods like attacker-in-the-middle (AiTM), token replay, and cross site scripting (XSS) to intercept and manipulate session tokens to bypass the intended MFA security measures.

Literature Related to the Project

Research has been conducted on session hijacking and the exploitation of vulnerable session management (Alahmad, Alkandari, & Alawadhi, 2022). The article provides an overview of broken authentication and session management attacks, which are ranked as a critical vulnerability in the OWASP Top 10. The article maintains a focus on common attack types, such as session hijacking, credential stuffing, and password spraying. Additionally, the article provides a structured analysis of how broken authentication and session management vulnerabilities are exploited and why security should be prioritized.

The usage of different literature to reinforce the methods of how these attacks are carried out was also central to research efforts (Baitha, & Vinod, 2018). This publication provides an analysis of different session hijacking attacks, focusing on how attackers can exploit session vulnerabilities to gain unauthorized access to sensitive information. The types of session hijacking attacks covered include active, passive, and hybrid session hijacking, each with its unique methods and implications. Additionally, various security measures to prevent session hijacking were also discussed, along with other recommendations to add additional layers of security.

Throughout the research for this project, many of the questions that were asked revolved around understanding where the risk derives from, as well as how to combat these risks once elucidated (Jain, Sahu, & Tomar, 2015). The paper analyzes the security risks that are caused by session hijacking. Session hijacking relies on misconfigurations of 'secure session management' to track user activity. Many of the vulnerabilities associated with session management involve cookies, which are the

primary exploitation vector in session hijacking attacks. The paper discusses how session IDs can be captured, which shows the unprecedented value attackers place in said session IDs. This analysis provided inspiration for the testing and development of a new way to combat session ID theft by brainstorming methods to implement security measures on different user-identifiable information.

Along with session hijacking, the project put an emphasis on exploitation in the context of using multi-factor authentication (MFA) as a security measure (Kharthik, Reddy, & Reddy, 2023). The article discusses the vulnerabilities of traditional password systems and highlights the importance of leveraging MFA as a stronger security measure, which has now become standard practice in most commercial environments. Due to the additional layer of defense provided by MFA, it is perceived as necessary in the effort to mitigate risks in scenarios like session hijacking. However, the report will show how these perceived ‘invincible’ security measures can be bypassed.

Many papers and articles expand on session hijacking with the use of XSS (Kumar, 2023). The publication provides a high-level explanation and step-by-step walk-through of the process of session hijacking using Cross-Site Scripting (XSS). Few articles and papers discuss methods to mitigate the effects of a session hijacking attack; however, leveraging the methodology may aid in determining a solution to the problem. Research published by the aforementioned Journal of Engineering Science and Technology is one of the few papers that list effective ways to fight against session hijacking. “1. Using HTTPS to ensure encryption of all messages on session traffic. 2. Setting HttpOnly attribute using Set-Cookie HTTP header to prevent XSS attack and

accessing cookie sessions from client side. 3. Validating all incoming data. 4. Rotating and regenerating of session ID after successful login” (Alahmad, Alkandari, & Alawadhi, 2022). While effective, each suggestion has a workaround or is impractical in a *commercial* setting.

Throughout the research, exploring the ways to mitigate session hijacking has been scattered (Letsoalo, & Ojo, 2016). While some research focused on web applications (Jain, Sahu, & Tomar, 2015), this paper presents a view from the side of networks on how to mitigate session hijacking attacks. The research looks to improve the detection of session hijacking. The model presented in this article combines three passive detection techniques: “De-authentication Frames Analysis (DFA), Received Signal Strength (RSS), and Round Trip Time (RTT)”. In hopes of leveraging the information to mitigate session hijacking attacks, utilizing the information provided by the article will help establish a basis for our solution in the project’s implementation.

In the past, MFA proved an effective way to help fortify a user’s account by adding an additional layer of security. However, the increase in session hijacking attacks proves MFA is not as protective as it once was (Mohammed, Dziyauddin, & Latiff, 2023). This paper reviews multi-factor authentication (MFA) systems, focusing on different approaches and implementations to improve user identity security. The paper also explores some aspects of the attack side of MFA and provides a full-view snapshot of current trends and countermeasures.

The research conducted for the purpose of the project is very limited in scope. However, with the aid of new research, an understanding of how session hijacking could be used in different scenarios and implementations has been curated (Muzammil, Bilal, Ajmal, Shongwe, & Ghadi, 2024). The publication analyzes critical web vulnerabilities, specifically focusing on Man-In-The-Middle (MITM) and session hijacking attacks. This article maintains relevance to the research on session hijacking as it examines the mechanics of these attacks. With this information, a deep understanding of how the attacks work allows for not only replicating said attacks but also establishing a foundation for remediating the vulnerability.

Initially, in order to gain a basic and foundational understanding of session hijacking and everything that the attack involves, research was conducted to learn related principles and methodology behind session hijacking (TheHackerNews, 2024). The article provides a very up-to-date analysis and introduction to modern session-hijacking techniques, which are highly relevant to research efforts on session-hijacking and bypassing MFA. The use of infostealers that extract session cookies from browsers ultimately enables attackers to bypass MFA and impersonate legitimate users.

Chapter III

METHODOLOGY

Introduction

This chapter describes and outlines the methodology and techniques used to analyze the inherent vulnerabilities of multifactor authentication (MFA) systems that allow session hijacking attacks to take place. The approach taken involves conducting controlled experiments in the simulation of real-world attack scenarios, which is done by testing the resilience of current MFA implementations against session hijacking techniques on a self-hosted network. The result is an initial baseline of the effect of the current security implementations, which was used to analyze and identify weaknesses in the authentication mechanisms. Once evaluated based on the effectiveness of the countermeasures, an understanding of the structure needed to properly implement security enhancements became evident. The following chapter details the design of the project, and includes the experimental setup, tools, and techniques used in the execution of further tests and any further research conducted. As well as developing a framework for a browser extension and secured backend database to protect sessions beyond the limits of MFA.

Design of the Project

While research into the protection of MFA systems from session hijacking attacks remains scarce, much can be learned through a simulated attack via a controlled system. Simply put, the first stage of the project is to create a baseline where one can determine what information is exposed during a session hijacking attack and how it can be used to get around MFA (Figure 1). The initial attack will be executed using Burp Suite and Firefox (Figure 2). As the first attack is to be a proof of concept, both the attack and victim commands will be run from the same computer. While not indicative of the real world, the only differences are the means of tricking the victim. This simple approach not only will prove that the attack can be done but makes it easier to stop and try another way in the case of a failed attempt (Figure 3).

After proving that a session hijacking attack can in fact compromise an MFA-based system, the execution of a more complex attack will be set up. To avoid continuously breaching Microsoft's TOS, a webserver was set up to host the domain of a new MFA-based login system. The usage of this server allows for even more transparency of the backend of the system and a more customized approach in the design of the protection framework, while also avoiding any potential future violations.

The project runs on a self-hosted domain, drcayden.com. Hosted on a Dell R610 ubuntu server instance, an node.js webserver running a simple login page and

dashboard for the site. Conditional access policies were established using the Office 365 Security Suite (Figure 4). Accounts were stored in a PostgreSQL database with bcrypt hashed passwords. MFA was enabled through the use of the Microsoft Authenticator App with speakeasy to create the Time-Based One-Time Password secret keys. Once a user account was created, they would be prompted to set up MFA on the first login to the site. A QR code would be displayed for the user to scan into the Microsoft Authenticator App. Once scanned, the secret key is stored in the database under the user and accessed for authentication of the Time-Based One-Time Passwords. Sessions are created, handled, and authenticated using express-sessions for node.js. When a user logs in to the site and authenticates, a session token or session ID is generated and stored in the PostgreSQL database under their username (Figure 5). This then creates a connect.sid cookie that is added to the user's browser that contains the signed session ID. When the user returns to the site, the session ID from the browser cookie is sent and matched to the session ID in the database. These session tokens are valid for 24 hours before they are removed from the database and will need to be recreated from the user re-authenticating their login. The secret key from the browser extension is also stored in the database under the user. This secret key is imported from the browser extension API and accessed upon login, and during continuous authentication.

Upon successful login, the site uses req.headers for node.js to grab a user's browser info and IP address. The IP address collected is then ran through the geolocation API "ipinfo" to get an approximate location for the user's login. All of this is stored in a separate PostgreSQL table for a user login history. Every time a user

attempts to log in, the data gathered is compared against the most recent login from the login history database. First, the new time is subtracted from the most recent login time to establish how many minutes it has been since the last login. Then the location data from both logins is used to find the distance between them, calculated using the haversine formula. Once the distance and time are known, speed is calculated. If the speed calculated is over 500 miles per hour, the travel is considered impossible, and a 403 status deny is sent to the user. This adds an extra layer of security to the site and the browser extension.

The browser extension works by having its own login that connects to the database of the webserver. Users log in to the browser extension with the same username and password as the site login. This allows for seamless integration. A user inputs a username and password into the browser extension, which will send a POST request to `dr cayden.com/api/extension-login`. The webserver then verifies the credentials through the users table by comparing the password hash column for the user in question. If the login is successful on the browser extension, a UUIDv4 token is generated and inserted into the extension table (username, token, issued). After this, the token is then returned into the extension. The token is saved in the browser's local storage (e.g. `chrome.storage.local`). This is what allows the token to persist across different browser sessions. Every time the browser extension is opened, it checks the storage location to see if a valid token exists. If a valid token exists, or if a user successfully logs into the extension, the GUI shows "Logged in as {username}.

Protecting sessions." This extension is integrated with our login server as when a user accesses this login, it checks to see if a valid extension token exists for the username

attempting to sign in or fetches it through the browser API or injected DOM postMessage. If the request for the token fails, or is incorrect, the user will not be logged in and we will be asked to authenticate again and check their browser extension to ensure it is enabled. This adds a third layer of authentication on top of MFA and session tokens.

Tools and Techniques Used

The foundational aspect of our prototype was the server that was utilized for the creation of the security solution. This was a Dell R610 server, with an Ubuntu operating system. This setup provided a reliable and controllable platform for deploying various services and tools. A Raspberry Pi was used to host the NGINX reverse proxy, which assisted in routing traffic and the establishment of port forwarding rules. The domain for the login page was purchased through Squarespace, and Cloudflare was used to manage the associated DNS records.

For security testing and attack testing, several tools were used, including Burp Suite, which was employed to retransmit session cookies and demonstrate the possibility of a session hijacking attack. Evilginx3, initially hosted on Digital Ocean before being removed due to abuse reports and was used to clone the login page via reverse proxy and forward credentials directly to Microsoft, making the phishing page nearly identical to the legitimate page.

The Microsoft Office 365 Security Suite was used in the configuration of several conditional access policies, and enabled Microsoft's experimental session hijacking

protections. Additionally, the enforcement of IT whitelisting, and review of monitor activity through audit and security logs allowed for a comprehensive overview of the hijacking attempts that were made, as well as the response of all implemented security measures.

Chapter IV

DISCUSSION AND RESULTS

Introduction

This chapter describes and outlines the results of our established methodology used in the creation of the prototype solution. The establishment of the associated variables, including the browser extension and the backend database, have been fully described regarding its effectiveness as a security solution, and the resulting outcome of its effectiveness in combatting the inherent vulnerabilities of multifactor authentication (MFA) and any associated session hijacking attempts. The resulting data and information displays that all conducted experimentation and implementation of the prototype have effectively fulfilled all design objectives while simultaneously acting as a solution to any prior problems regarding session hijacking in the context of MFA.

Results and Discussion

To assess the vulnerabilities of MFA systems, a series of session hijacking attacks was conducted. The initial proof-of-concept attack on a Microsoft login demonstrated that an active session could successfully be hijacked, bypassing MFA. While the attack utilized Burp Suite's proxy service rather than an attack vector like phishing. It still helped in confirming that MFA could be bypassed via session hijacking.

After creating a self-hosted domain, dr cayden.com, that also utilized MFA – a secondary session hijacking attack was conducted on it. Like the first attack, Burp Suite was utilized to steal the session cookies. Mimicking the steps from the initial attack, the

active session was successfully hijacked. This showed that the created domain would be a valid substitution for a Microsoft account, as both were affected in the same way.

After creating the browser extension, testing shifted from proving that sessions could be bypassed to proving that sessions could be protected beyond the means of MFA. The browser, in conjunction with a secured backend database, protects a user's active session by verifying machine and browser information as well as contextual data on each sign-in. During this testing phase, a successful login was initiated. On the backend, the browser grabbed and sent machine and browser information to tie to the created session. The tab was closed, and the user reopened the drcayden domain on the same machine and was allowed to sign back in as normal. This showed that even with the updated security policies, valid users are still able to log into their systems as normal.

Next, the session cookies were taken from the active session via Firefox's cookie editor extension. This was used to simulate an attacker gaining access to the cookies via a phishing or XSS attack. The stolen cookies were then inputted in a private browser that was not tied to the active session. After opening the drcayden domain, this secondary user was not able to log in (Figure 6). While they had the session cookies, the browser and machine data that was tied to the active session did not match the user's data. This proved that the updated security policies worked to protect an active session from unauthorized access.

Objective 1: Understand how session hijacking occurs

By executing controlled session hijacking scenarios utilizing Burp Suite and Firefox's cookie editor, the mechanics of session hijacking were observed. It showed that after a valid session token was captured, MFA no longer played a role in maintaining session integrity.

Objective 2: Perform a simulated session hijack against a known domain and identify weaknesses using a domain with MFA implementation

Successful session hijacking attacks were made against both a self-hosted web domain secured with MFA as well as a Microsoft domain secured with MFA. The attacks confirmed that with access to an active session's tokens, one can successfully log in anywhere.

Objective 3: Identify weaknesses in OAUTH Multifactor Authentication (MFA)

Through the successful attacks, it was revealed that while OAUTH-based MFA works well to initially authenticate a user, they rely on tokens with limited to no context. Regardless of the IP address, browser data, or geolocation information, a session will remain valid.

Objective 4: Identify protection measures to mitigate and/or prevent session tokens from being used from different systems

Many strategies were explored, geolocation tracking, browser and device fingerprinting, and backend validation. These three strategies used in conjunction are used as the basis of the framework to protect a system against session hijacking and token abuse.

Objective 5: Test the implementations of all findings to ultimately increase the security posture of any system implementation utilizing MFA

A browser extension and secured backend database framework were developed to log browser and machine data and tie it to an active session. On each attempt to log into a given session, the data is cross-checked with what is saved in the database.

Testing showed that the framework was able to effectively prevent a hijacked session.

Chapter V

CONCLUSION AND FUTURE WORK

Conclusion

Our research and technical analysis of session hijacking demonstrates that Multi-Factor Authentication alone is insufficient in preventing session hijacking once a user has already authenticated. Session hijacking takes advantage of the lack of verification for a live session, granting unprecedented access to attackers. While solutions exist, aimed at preventing unauthorized access to a user's session.

Many current solutions fail at protecting an already active session from unauthorized access and reuse. By introducing a browser extension and a backend verification layer, our solution provides real-time protection and session verification based on a user and their session behavioral context. This solution adds verification on the hardware level, making it extremely difficult for attackers to emulate.

Future work

This framework offers a starting point for improving session security beyond MFA. Organizations can tailor and expand it to fit their specific systems, such as adding behavior-based detection or broader platform support. As threats evolve, this flexible design allows for ongoing upgrades to stay ahead of potential attacks. To further develop and implement our solution, future enhancements and features may be made to further optimize the effectiveness of the prototype solution.

The creation of an administration dashboard would allow users, either from in a personal or enterprise setting, the ability to add an extra level of configuration of the settings to better fit business or security needs. In the same scope, the prototype would work seamlessly with the potential to connect a Security Information & Event Manager (SIEM) solution to further increase the security capabilities of the browser extension.

As a framework, the prototype solution provides for the ability to provide security at any level or in any setting. Because of this, the foundational application of the prototype is a simplified and straightforward implementation. However, future implementations may include a more simplified and basic approach to security in order to further provide for even the most basic security needs of the user. Finally, the information provided to the user may be optimized to provide more details on events that take place during the utilization of the solution. A more detailed reporting dashboard may be implemented in the future to provide users with an extra level of information or data points. With this increased level of knowledge, users would be able to pinpoint potential areas of concern and fit the needs of the user in a more efficient way.

REFERENCES

- Alahmad, M., Alkandari, A., & Alawadhi, N. (2022, April). Survey of broken authentication and session management of web application vulnerability attack. *Journal of Engineering Science and Technology*, 17(2), 504–520.
https://jestec.taylors.edu.my/Vol%2017%20Issue%202%20April%20%202022/17_2_04.pdf
- Baitha, A. K., & Vinod, S. (2018). Session hijacking and prevention technique. *International Journal of Advanced Research in Computer Science*, 9(3), 455–459.
https://www.researchgate.net/publication/325117343_Session_Hijacking_and_Prevention_Technique
- Jain, V., Sahu, D. R., & Tomar, D. S. (2015). Session hijacking: Threat analysis and countermeasures. *International Journal of Computer Applications*, 115(8), 28–33.
https://www.researchgate.net/publication/277307339_Session_Hijacking_Threat_Analyses_and_Countermeasures
- Kharthik, B., Reddy, K., & Reddy, B. I. (2023). A comparative analysis of various multifactor authentication mechanisms. *International Journal of Computer Sciences and Engineering*, 11(3), 105–113.
https://www.researchgate.net/publication/368830831_CSEIT1835225_A_Comparative_Analysis_of_Various_Multifactor_Authentication_Mechanisms
- Kumar, R. (2023). Session hijacking with XSS using local server. *Medium*.
<https://medium.com/@rk702370/session-hijacking-with-xss-using-local-server-f9a8b3a2feeb>

- Letsoalo, E., & Ojo, S. (2016). A model to mitigate session hijacking attacks in wireless networks. *2016 International Conference on Advances in Computing and Communication Engineering (ICACCE)*, 35–40. <https://ieeexplore.ieee.org/document/8417199>
- Mohammed, A. H. Y., Dziyauddin, R. A., & Latiff, L. A. (2023). Current multi-factor of authentication: Approaches, requirements, attacks, and challenges. *International Journal of Computer Sciences and Engineering*, 11(4), 120–135.
https://www.researchgate.net/publication/367987835_Current_Multi-factor_of_Authentication_Approaches_Requirements_Attacks_and_Challenges
- Muzammil, M. B., Bilal, M., Ajmal, S., Shongwe, S. C., & Ghadi, Y. Y. (2024). Unveiling vulnerabilities of web attacks considering man in the middle attack and session hijacking. *2024 IEEE International Conference on Cybersecurity and Internet of Things (ICCIoT)*, 67–73. <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10381703>
- TheHackerNews (2024). Session hijacking 2.0 — The latest way that attackers are bypassing MFA. *The Hacker News*. <https://thehackernews.com/2024/09/session-hijacking-20-latest-way-that.html>